Modifications

Page No:
Date:

Title: Mirror image of Binary Tree

Problem statement:
To develop a C++ Program to find the mirror image of a binary tree by swapping the left and right children of every node. The Program displays the original and mirrored tree traversals side by side, mirrors the tree twice to verify that the original structure is restored, and compares the no. of node and height before and after mirroring.

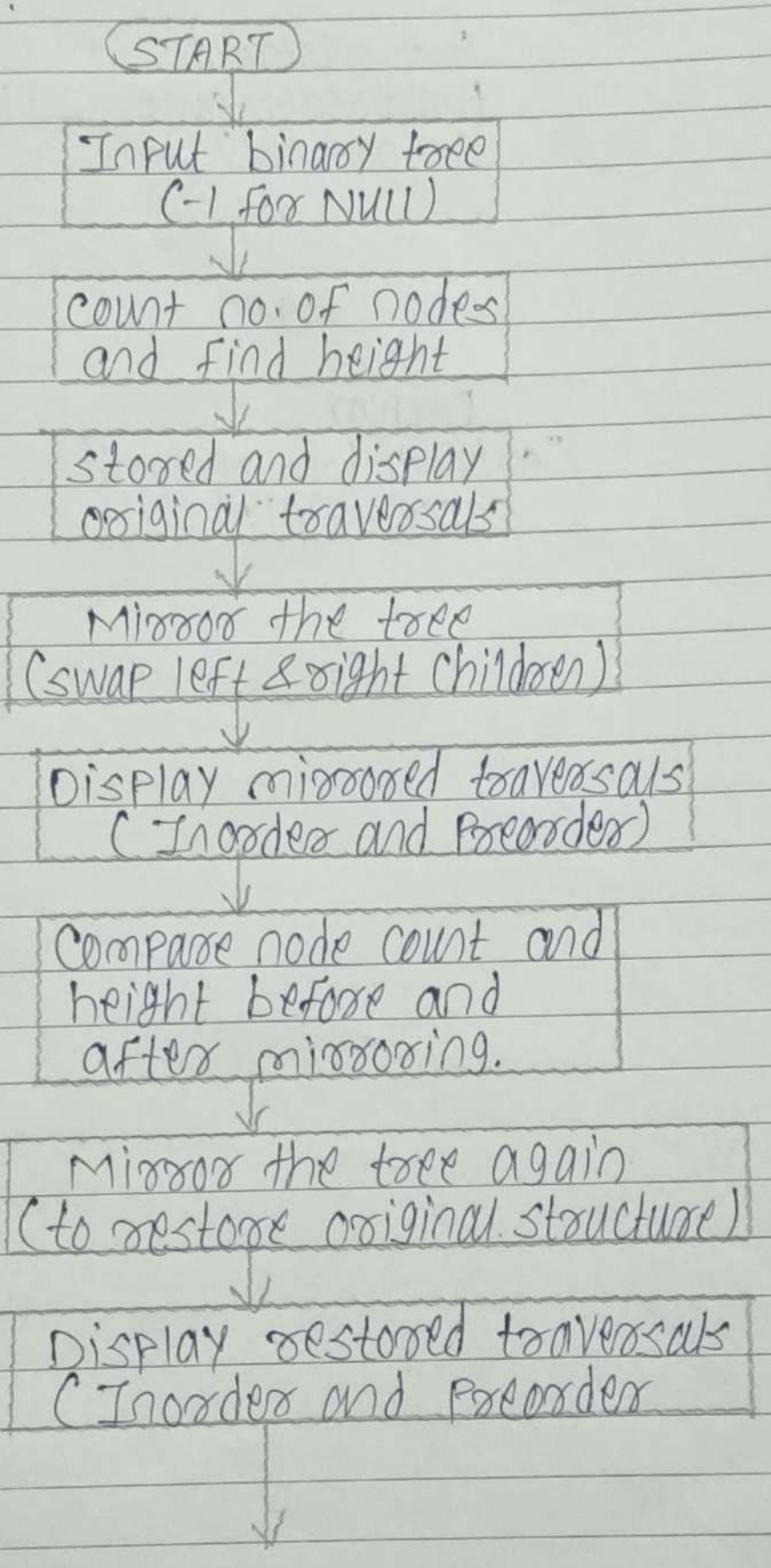
Objectives:
To create a binary tree using user input.
To display the original and mirrored traversals side by side.
To find the mirror image by swapping left and right children.
To mirror the tree twice and verify that the original structure is restored.
To compare the node count and height before and after morning.

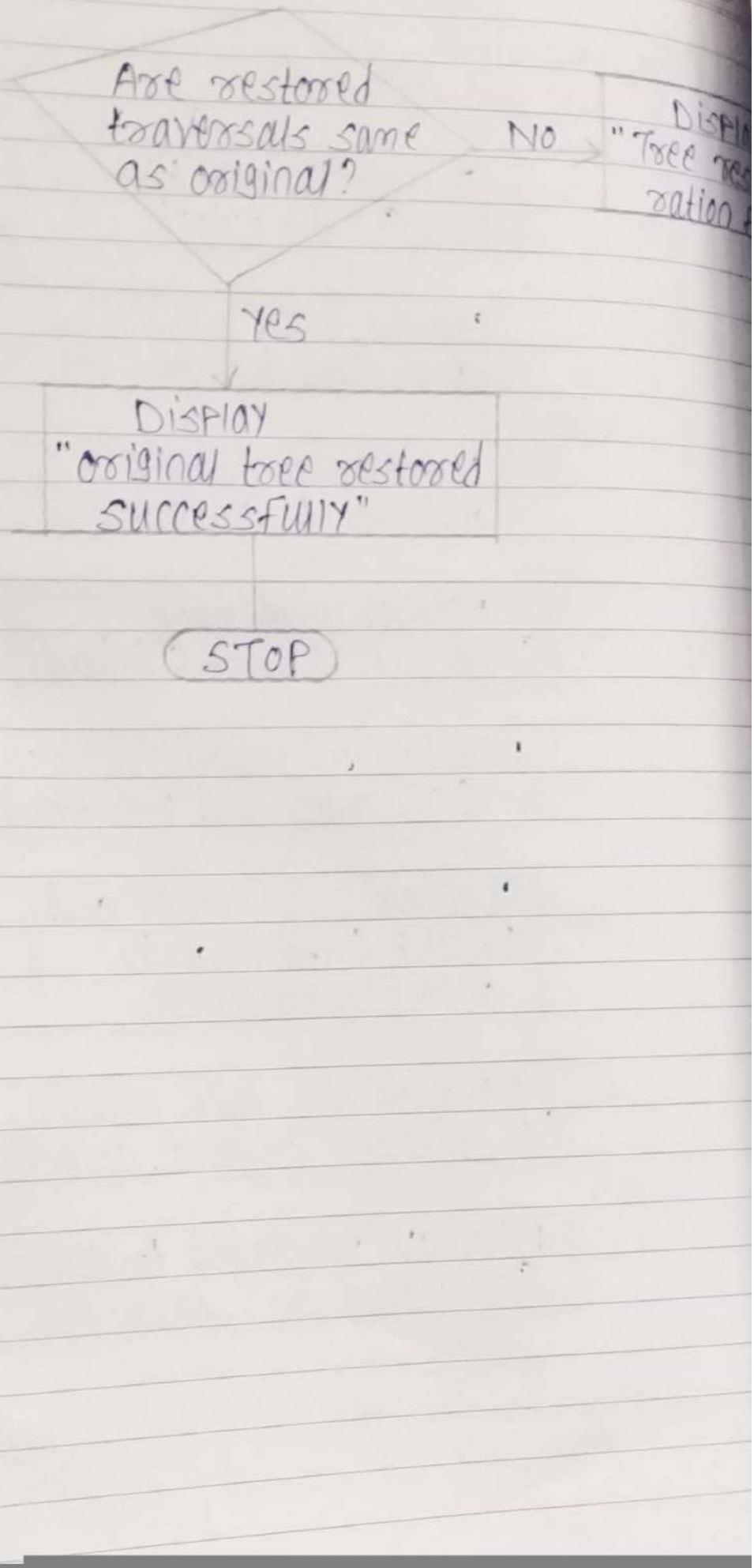
Algorithm:
Start
Create a binary tree using user input.
Enter -1 to represent an empty node.
Count the total number of nodes in the original tre
Calculate the height of the original tree.

5) Display the original inorder traversal.
6) Mirror the tree by swapping the left and r children of each node.
7) Display the mirrored inorder traversal.
8) Compare the node counts and heights of the ori and mirrored trees.
9) Mirror the tree again to restore the original
10) Display the restored inorder traversal and ve that it matches the original traversal.
11) Stop.

Page No:
Date:

1) Flowchart:




Are restored traversals same as original?
No
Yes
Display "original tree restored successfully"
STOP

```
#include <iostream>
#include <string>
using namespace std;

struct Node {
   int data;
   Node *left, *right;
};

Node* createTree() {
   int x;
   cin >> x;

   if (x == -1)
      return NULL;

   Node* n = new Node;
   n->data = x;
   n->left = createTree();
   n->right = createTree();

   return n;
}

void inorder(Node* root, string &s) {
   if (root == NULL) return;

   inorder(root->left, s);
   s += to_string(root->data) + " ";
   inorder(root->right, s);
}
```

```
void preorder(Node* root, string &s) {
    if (root == NULL) return;

    s += to_string(root->data) + " ";
    preorder(root->left, s);
    preorder(root->right, s);
}

void mirror(Node* root) {
    if (root == NULL) return;

    Node* temp = root->left;
    root->left = root->right;
    root->right = temp;

    mirror(root->left);
    mirror(root->right);
}

int countNodes(Node* root) {
    if (root == NULL) return 0;

    return 1 + countNodes(root->left)
             + countNodes(root->right);
}

int height(Node* root) {
    if (root == NULL) return 0;

    int l = height(root->left);
    int r = height(root->right);

    return 1 + (l > r ? l : r);
```

```
}

int main() {
    cout << "Enter tree in preorder (-1 for NULL):\n";
    Node* root = createTree();

    string originalIn, originalPre;
    inorder(root, originalIn);
    preorder(root, originalPre);

    int nodes = countNodes(root);
    int h = height(root);

    cout << "\nOriginal Inorder: " << originalIn;
    cout << "\nOriginal Preorder: " << originalPre;

    mirror(root);

    string mirrorIn, mirrorPre;
    inorder(root, mirrorIn);
    preorder(root, mirrorPre);

    cout << "\n\nMirrored Inorder: " << mirrorIn;
    cout << "\nMirrored Preorder: " << mirrorPre;

    cout << "\n\nOriginal Nodes: " << nodes;
    cout << "\nMirrored Nodes: " << countNodes(root);
    cout << "\nOriginal Height: " << h;
    cout << "\nMirrored Height: " << height(root);

    mirror(root);

    string restoredIn, restoredPre;
```

```
    inorder(root, restoredIn);
    preorder(root, restoredPre);

    cout << "\n\nRestored Inorder: " << restoredIn;
    cout << "\nRestored Preorder: " << restoredPre;

    if (restoredIn == originalIn &&
        restoredPre == originalPre)
        cout << "\nOriginal tree restored successfully.";
    else
        cout << "\nTree restoration failed.";

    return 0;
}
```

```
Enter tree in preorder (-1 for NULL):
1
2
4
-1
-1
5
-1
-1
3
-1
-1

Original Inorder: 4 2 5 1 3
Original Preorder: 1 2 4 5 3

Mirrored Inorder: 3 1 5 2 4
Mirrored Preorder: 1 3 2 5 4

Original Nodes: 5
Mirrored Nodes: 5
Original Height: 3
Mirrored Height: 3

Restored Inorder: 4 2 5 1 3
Restored Preorder: 1 2 4 5 3
Original tree restored successfully.
--------------------------------
Process exited after 38.01 seconds with return value 0
Press any key to continue . . .
```